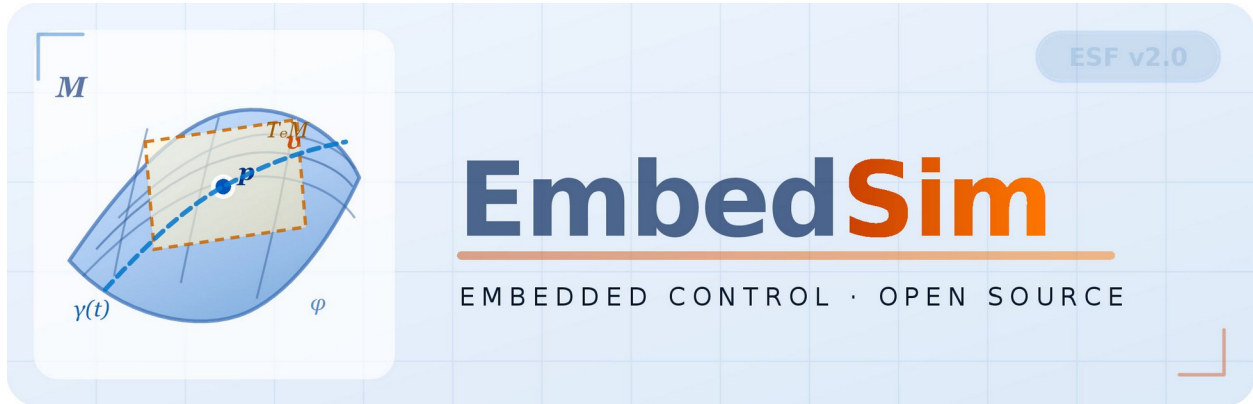




Deep Architecture Reference

Python -- Cython -- C Code Generation -- FMU / Modelica

Revision 3.0 -- March 2026 -- MISRA C:2012 -- ISO 26262 ASIL

What this document covers: The complete internal mechanics of EmbedSim -- how the simulation engine traverses block graphs, how C names are derived from Python metadata, how types flow from Python attributes to generated struct fields, how Cython bridges the Python and C worlds, how FMU plants are wrapped and connected, and how three concrete examples illuminate each mechanism. Read the code alongside this document; the code is the proof.

Part 1 Architecture: The Four-Layer Stack

1.1 The Central Principle

EmbedSim is built around one inversion of the usual embedded workflow. Normally you write C, flash it, probe it with a debugger, and reverse-engineer what the algorithm is doing. EmbedSim inverts this: you write the algorithm in Python first, simulate it to full numerical correctness, and then have the framework translate it to C automatically.

The consequence is that the Python simulation and the MCU firmware are always identical in behaviour. Not approximately identical -- literally the same algorithm, translated mechanically. If the Python simulation is correct, the MCU firmware is correct. The only remaining risk is in the translation mechanics themselves, which is why those mechanics are documented here in full.

1.2 Four Layers, Four Roles

The stack has exactly four layers. Each has a single, well-defined role that does not overlap with any other layer.

Layer	Technology	Single responsibility	What you hand-write
1 -- Python	NumPy / pure Python	Specify and verify the algorithm	compute_py() method body
2 -- Cython	.pyx + C extension	Compile the same algorithm for performance	cdef extern declarations, cpdef wrappers
3 -- Generated C	MISRA StepGenerator	Translate verified Python to MCU production code	Nothing: it is emitted automatically
4 -- FMU / Modelica	OpenModelica + FMPy	Provide physics-correct plant models	Modelica equations (.mo file)

The fundamental invariant: No information flows from Layer 3 back to Layer 1. The C code is always derived from the Python model, never the reverse. A simulation file that contains a C type name, a C function name, or a C string literal is carrying a code-generator bug in disguise. Every name in generated C is derived programmatically from Python block metadata.

1.3 How Information Flows Between Layers

Understanding EmbedSim requires understanding how data and metadata flow between layers. There are two distinct flows: the runtime signal flow (what happens during simulation) and the code-generation metadata flow (what happens when you call generate_step()).

Runtime signal flow (simulation time)

```
EmbedSim.run()
for each step k:
    _compute_all_blocks(t, dt)
    for each block in topological order:
        input_values = [upstream_block.output for upstream in block.inputs]
        block.compute(t, dt, input_values)
        -> routes to compute_c() or compute_py()
        -> updates block.output (a VectorSignal wrapping a np.float32 array)
```

Signal type at every boundary:

```
VectorSignal.value = np.float32 array    <- Python side
float[:,1] memoryview                  <- Cython side (same buffer, no copy)
real32_T* pointer                      <- C side (same buffer, no copy)
```

Code-generation metadata flow (generate_step() time)

```
cg_end.generate_step(cg_start, output_dir, dt_hz)
StepGenerator.__init__()
    DFS backward from cg_end to cg_start
    collect ordered block list

StepGenerator.generate_header()
    collect INPUT_NAMES from cg_start upstream blocks -> EmbedSim_Input_T fields
    collect OUTPUT_NAMES from cg_end upstream blocks -> EmbedSim_Output_T fields
    #include all C_HEADERS from all blocks in region
    emit EMBEDSIM_DT = 1/dt_hz

StepGenerator.generate_source()
    emit static state declarations: block.state_struct block.name + "_state"
    emit EmbedSim_Init(): call block.init_func for each stateful block
    emit EmbedSim_Step(): call _emit_block() for each block in order
```

1.4 Naming Convention -- How Every C Name is Derived

Every C identifier in generated code is derived mechanically from Python attributes. There are no magic strings, no hardcoded names, no manual mappings. Understanding the derivation rules is the key to understanding what the generated code will look like.

Generated C identifier	Derived from	Example
State variable name	block.name + "_state"	dfc -> dfc_state
State struct type	block.state_struct	DFC_State_T
Init function call	block.init_func + "(&" + name + "_state, ...)"	DFC_Controller_Init(&dfc_state, ..)
Step function call	block.step_func + "(&" + name + "_state, ...)"	DFC_Controller_Step(&dfc_state, ...)
y[] buffer name	"y_" + block.name	y_dfc, y_svpwm_pack
u[] buffer name	"u_" + block.name	u_dfc, u_svpwm
Input struct field	INPUT_NAMES[i]	in->omega_ref_mech, in->theta_m
Output struct field	OUTPUT_NAMES[i]	out->ta, out->sector
Init literal argument	str(getattr(block, C_INIT_ARGS[i]))	41.88790205f (from block.omega_target)
Header #include	C_HEADERS[i].lower()	#include "embed_sim_dfc_controller.h"
Timestep macro	"EMBEDSIM_DT" value = 1/dt_hz as f-literal	#define EMBEDSIM_DT (0.00005000f)

The lowercase rule: StepGenerator calls .lower() on every C_HEADERS entry. This enforces consistent lowercase include paths regardless of how the Python string was written. A block that declares

C_HEADERS = ["Embed_Sim_DFC_Controller.H"] will generate #include "embed_sim_dfc_controller.h" -- always matching the actual filesystem convention.

Part 2 Simulation Engine: DFS, Algebraic Loops, and Execution Order

2.1 The Block Graph

Every EmbedSim block subclasses `VectorBlock`. Blocks are connected by the `>>` operator, which appends the right-hand block to the left-hand block's `successors` list and appends the left-hand block to the right-hand block's `inputs` list. At this stage no computation happens -- the graph is a pure Python object graph.

```
sin_src >> loop_sum >> loop_gain >> loop_out
# Equivalent to:
# sin_src.successors.append(loop_sum); loop_sum.inputs.append(sin_src)
# loop_sum.successors.append(loop_gain); loop_gain.inputs.append(loop_sum)
# loop_gain.successors.append(loop_out); loop_out.inputs.append(loop_gain)

# >> is associative and supports fan-out:
dfc >> svpwm_pack >> svpwm >> cg_end # chain
dfc >> svpwm_pack                    # fan-out (two >> calls to same target)
```

2.2 Topological Sort -- Two-Pass DFS

When `EmbedSim.__init__()` is called, it runs `traverse_blocks_from_sinks_with_loops()` to produce a flat ordered list `blocks_order`. This list is the execution order for every simulation step. The algorithm is a depth-first search (DFS) from every sink block, run in two passes.

Pass 1: find_loop_breakers()

Walk the entire graph from every sink, following `block.inputs` recursively. Collect every block whose class inherits from `LoopBreaker` into the set `loop_breakers`. Identification is $O(1)$ via `block.is_loop_breaker` flag -- no `isinstance()` on every edge.

Pass 2: dfs() -- post-order with loop handling

```
def dfs(block, visiting, visited, loop_breakers, blocks_order):
    if block in visited:
        return
    if block in visiting:
        # back-edge: check if it is a LoopBreaker
        if block not in loop_breakers:
            raise ValueError(f"Algebraic loop detected at: {block.name}")
        # LoopBreaker: insert NOW, stop recursion (back-edge is cut)
        if block not in visited:
            blocks_order.append(block)
            visited.add(block)
        return

    visiting.add(block)
    if block not in loop_breakers:
        for inp in block.inputs:
            dfs(inp, visiting, visited, loop_breakers, blocks_order)
    # post-order: append after all inputs
    blocks_order.append(block)
    visiting.discard(block)
    visited.add(block)
```

The counter-intuitive result: a `VectorDelay` block is appended to `blocks_order` BEFORE its upstream source. In the closed-loop DFC example, `motor_delay` is placed first in the execution list -- before `ctrl_packer`, before `dfc`, before `motor`. This is correct: `motor_delay` outputs `motor(k-1)`, which was stored during the previous step. It does not need `motor` to have run at step `k`.

2.3 Runtime Execution: `_compute_all_blocks()`

Every simulation step calls `_compute_all_blocks(t, dt)`. This iterates `blocks_order` exactly once, left to right:

```
def _compute_all_blocks(self, t, dt):
    # Step A: pre-initialise LoopBreakers whose output is None (t=0 only)
    for block in self.blocks_order:
        if isinstance(block, LoopBreaker) and block.output is None:
            block.output = block.get_loop_breaking_output()
            # VectorDelay: returns VectorSignal(initial)

    # Step B: forward pass
    for block in self.blocks_order:
        input_values = []
        for inp in block.inputs:
            if inp.output is not None:
                input_values.append(inp.output)
            elif isinstance(block, LoopBreaker):
                input_values.append(VectorSignal([0.0])) # silent: by design
            else:
                self.logger.warning(f"zero-fallback: {block.name} <- {inp.name}")
                input_values.append(VectorSignal([0.0]))
        block.compute(t, dt, input_values) # updates block.output
```

Why zero-fallback exists: At $t=0$, `motor_delay` runs first. Its upstream (motor) has `output=None` because motor has not yet run this step. The engine substitutes `[0.0]` and continues. `motor_delay` ignores this substitute -- it returns `last_output=initial` regardless of what the current input is. The warning fires only at $t=0$ and disappears at $t=1$ because `motor.output` is then set from the previous step.

2.4 ODE Integration

After `_compute_all_blocks()` completes for a step, the engine advances stateful dynamic blocks (integrators, state-space models) through the chosen ODE solver. EmbedSim supports three solvers:

Solver	Order	Cost	Use case
ODESolver.EULER	1	1x <code>_compute_all_blocks</code>	Fastest; acceptable for AURIX hardware-matched simulations at 20 kHz
ODESolver.HEUN	2	2x <code>_compute_all_blocks</code>	Recommended default: predictor-corrector, 2nd-order accuracy
ODESolver.RK4	4	4x <code>_compute_all_blocks</code>	Highest accuracy; 4x cost; triggers 4x zero-fallback warnings at $t=0$

VectorDelay is not an ODE state. Its state update (latching current input as `last_output`) happens inside `compute_py()` at Step B. The ODE integrator sees `VectorDelay` as a stateless block. This is correct: the z^{-1} operation is a sample-and-hold, not an integral.

2.5 VectorSignal -- The Signal Carrier

`VectorSignal` is a thin wrapper around a `np.float32` array with an optional name tag. It is the type of every `block.output` and every element of `input_values`. The `float32` dtype is enforced at construction: passing a `float64` array quietly truncates to `float32`. The `DEFAULT_DTYPE = np.float32` constant is imported throughout the codebase to avoid any accidental `float64` construction.

```
class VectorSignal:
    def __init__(self, value, name=""):
        # value is always stored as float32
        if isinstance(value, np.ndarray):
            self.value = value.astype(np.float32)
        else:
            self.value = np.array(value, dtype=np.float32)
        self.name = name

# Usage at block output:
self.output = VectorSignal(
    np.array([v_alpha, v_beta], dtype=DEFAULT_DTYPE),
    self.name
)
```

Part 3 VectorBlock: Metadata, Routing, and the C Contract

3.1 VectorBlock Base Class

Every EmbedSim processing block subclasses `VectorBlock`. The base class provides the `>>` wiring operator, the `use_c_backend` routing flag, the `compute()` dispatcher, and all the class-level attributes that the code generator reads.

```
class VectorBlock:
    # ---- CodeGen attributes (read by StepGenerator) ----
    PYX_FILE      = None      # str: path to .pyx wrapper (triggers PYXInspector)
    C_SOURCES     = []        # list of .c files
    C_HEADERS     = []        # list of .h files (auto-lowercased by StepGenerator)
    NUM_INPUTS    = 1         # 0 for source blocks (no upstream)
    OUTPUT_SIZE   = 1         # number of scalar outputs in y[]
    C_INIT_ARGS   = []        # instance attribute names -> typed Init() literals
    C_CUSTOM_EMIT = None      # str: verbatim C emitted for non-standard ABI
    C_CODEGEN_EXCLUDE = False # True -> never enters StepGenerator

    # ---- Boundary struct attributes ----
    INPUT_NAMES   = []        # field names for EmbedSim_Input_T
    INPUT_KEEP    = []        # indices into upstream y[] for each INPUT_NAMES entry
    C_INPUT_TYPES = {}        # dict: field -> C type (default: real32_T)
    OUTPUT_NAMES  = []        # field names for EmbedSim_Output_T
    OUTPUT_KEEP   = []        # indices into y[] for each OUTPUT_NAMES entry
    C_OUTPUT_TYPES = {}       # dict: field -> C type (default: real32_T)

    # ---- Auto-populated by PYXInspector (if PYX_FILE is set) ----
    step_func     = None      # str: C step function name
    state_struct  = None      # str: C state struct type name
    init_func     = None      # str: C init function name

    # ---- Topology ----
    TOPO_CATEGORY = "block"   # "plant", "utility", "source", "sink", "codegen"
    output_label  = ""        # displayed in topology printer
```

3.2 The compute() Dispatcher

The `compute()` method is the single call point for the simulation engine. Every `VectorBlock` subclass that has a C backend implements both `compute_py()` and `compute_c()`. The dispatcher routes based on `use_c_backend` and `_wrapper` is not `None`:

```
# In DFControllerBlock (and all blocks with C backends):
def compute(self, t, dt, input_values=None):
    if self.use_c_backend and self._wrapper is not None:
        return self.compute_c(t, dt, input_values)
    return self.compute_py(t, dt, input_values)

# The C backend path: compute_c()
def compute_c(self, t, dt, input_values=None):
    # 1. Unpack upstream VectorSignal into a flat np.float32 array
    inputs = np.zeros(5, dtype=np.float32)
    inputs[0] = float(u[0]) # omega_ref_mech [rad/s]
    inputs[1] = float(u[1]) # theta_m [rad]
    inputs[2] = float(u[2]) # ia [A]
    inputs[3] = float(u[3]) # ib [A]
    inputs[4] = float(u[4]) # ic [A]

    # 2. Call through Cython wrapper -- zero copy (float[:,1] shares buffer)
    self._wrapper.set_inputs(inputs) # float[:,1] memoryview
    self._wrapper.compute(float(dt)) # calls DFController_Step() in C
    outputs = self._wrapper.get_outputs() # returns np.float32 array

    # 3. Wrap in VectorSignal and store
```



```
self.output = VectorSignal(outputs, self.name)
return self.output
```

3.3 PYXInspector -- Automatic Metadata Population

PYXInspector is a static analysis tool that reads .pyx wrapper files as plain text using regular expressions. It runs at class-definition time via Python's `__init_subclass__` hook -- meaning the metadata is populated when the class is imported, not when an instance is created.

How `__init_subclass__` hooks PYXInspector

```
# In VectorBlock (base class):
def __init_subclass__(cls, **kwargs):
    super().__init_subclass__(**kwargs)
    if cls.PYX_FILE is not None:
        PYXInspector.populate(cls) # runs immediately when class body executes

# PYXInspector.populate() scans the .pyx file and fills in:
# cls.step_func    <- function name containing STEP, COMPUTE, or UPDATE
# cls.init_func    <- function name containing INIT
# cls.state_struct <- first ctypedef struct name found
# Only writes if the attribute is currently None (explicit declaration wins)
```

What PYXInspector reads from the .pyx file

```
# Pattern 1: ctypedef struct -> state_struct
# Matches: "ctypedef struct DFC_State_T:"
PATTERN_STRUCT = re.compile(r'ctypedef struct+(w+)')

# Pattern 2: cdef extern function -> step_func or init_func
# Matches: "void DFC_Controller_Step(DFC_State_T *s, ...)"
PATTERN_FUNC = re.compile(r'(?:(void|int|float)s+(w+)s*(')

# Classification:
# INIT in uppercase name    -> init_func
# STEP, COMPUTE, or UPDATE -> step_func
# RESET                    -> reset_func

# Example .pyx file excerpt:
cdef extern from "embed_sim_dfc_controller.h":
    ctypedef struct DFC_State_T:           # -> state_struct = "DFC_State_T"
        pass
    void DFC_Controller_Init(DFC_State_T *s, float dt_s) nogil # -> init_func
    void DFC_Controller_Step(DFC_State_T *s, ...) nogil        # -> step_func
```

Explicit declarations always win: if **DFCControllerBlock** explicitly declares `step_func = "DFC_Controller_Step"` in the class body, PYXInspector will not overwrite it even if it scans the .pyx and finds a different function name. This means that blocks with unusual naming conventions can override the auto-detection without changing the .pyx file.

3.4 C_INIT_ARGS -- Typed Literal Injection

C_INIT_ARGS is a list of instance attribute names. At code-generation time, StepGenerator reads the live value of each named attribute from the block instance and emits it as a typed C float literal with the 'f' suffix, satisfying MISRA Rule 7.2.

```
# Block class:
class SpeedRampBlock(VectorBlock):
    C_INIT_ARGS = ["omega_target", "ramp_time"]
```

```

# Block instance in simulation file:
speed_ref = SpeedRampBlock("speed_ref",
                           omega_target = 41.88790205, # 400 RPM in rad/s
                           ramp_time    = 0.5)

# Generated Init() call in EmbedSim_step.c:
SpeedRamp_Init(&speed_ref_state, 41.88790205f, 0.50000000f);
#               ^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^
#               from speed_ref.omega_target speed_ref.ramp_time

# For DFControllerBlock with C_INIT_ARGS = ["dt_s"]:
DFC_Controller_Init(&dfc_state, 0.00005000f);
#               ^^^^^^^^^^^^^
#               from dfc.dt_s = 50e-6

```

Why C_INIT_ARGS works this way: The block instance exists and has been run during simulation by the time `generate_step()` is called. All instance attributes have their final, verified values. StepGenerator reads them with `getattr(block, attr_name)` and formats them as f-suffixed float literals. This means the `Init()` call always matches the simulation exactly -- there is no separate 'config file' that could drift from the simulation parameters.

Part 4 The Cython Layer: Bridging Python and C

4.1 Why Cython

Cython is the bridge layer between Python and C. It serves two distinct purposes in EmbedSim: (1) it provides a high-performance path for blocks that need real-time execution speed during simulation, and (2) it allows the Python simulation to call the exact same C function that will run on the MCU, verifying numerical identity between simulation and production code.

The `float[:,1]` contract is the critical detail. A Cython `float[:,1]` memoryview is a C-contiguous buffer of 32-bit floats. When a Python `np.float32` array is passed to a `float[:,1]` parameter, Cython validates the dtype and layout and provides a direct pointer to the buffer's memory -- no copy. When a `np.float64` array is passed, Cython does NOT raise an error. It silently succeeds, but the internal buffer remains at zero because the element size (8 bytes) does not match what the C function expects (4 bytes). Every output reads 0.5. The motor hums but does not rotate.

4.2 Standard Cython Wrapper Structure

Every EmbedSim Cython wrapper follows a fixed three-method pattern. The pattern is identical across all blocks; only the struct types and function names change.

```
# cython: language_level=3
# cython: boundscheck=False
# cython: wraparound=False

cimport numpy as cnp
import numpy as np

# Step 1: declare the C types and functions to Cython
cdef extern from "embed_sim_dfc_controller.h":
    ctypedef struct DFC_State_T:
        pass # Cython only needs the type, not fields
    ctypedef struct DFC_Input_T:
        float omega_ref_mech
        float theta_m
        float ia, ib, ic
    ctypedef struct DFC_Output_T:
        float v_alpha, v_beta

    void DFC_Controller_Init(DFC_State_T *s, float dt_s) nogil
    void DFC_Controller_Step(DFC_State_T *s,
                            const DFC_Input_T *u,
                            float dt,
                            DFC_Output_T *y) nogil

# Step 2: define the Python-callable wrapper class
cdef class DFControllerWrapper:
    cdef DFC_State_T _state # C struct allocated on the heap by Cython
    cdef float[5] _u_buf # fixed-size input buffer
    cdef float[2] _y_buf # fixed-size output buffer

    def __init__(self, float dt_s):
        DFC_Controller_Init(&self._state, dt_s)

    # set_inputs: receives np.float32 array from Python, zero-copy via float[:,1]
    cpdef void set_inputs(self, float[:,1] u):
        # CRITICAL: float[:,1] not double[:,1]
        # If caller passes float64 array, Cython silently zeroes the buffer
        self._u_buf[0] = u[0] # omega_ref_mech
        self._u_buf[1] = u[1] # theta_m
        self._u_buf[2] = u[2] # ia
        self._u_buf[3] = u[3] # ib
        self._u_buf[4] = u[4] # ic
```

```

# compute: calls C function with GIL released
cpdef void compute(self, float dt):
    cdef DFC_Input_T u_in
    cdef DFC_Output_T y_out
    u_in.omega_ref_mech = self._u_buf[0]
    u_in.theta_m        = self._u_buf[1]
    u_in.ia              = self._u_buf[2]
    u_in.ib              = self._u_buf[3]
    u_in.ic              = self._u_buf[4]
    with nogil:
        DFC_Controller_Step(&self._state, &u_in, dt, &y_out)
    self._y_buf[0] = y_out.v_alpha
    self._y_buf[1] = y_out.v_beta

# get_outputs: returns np.float32 array to Python
cpdef cnp.ndarray get_outputs(self):
    cdef cnp.ndarray y = np.empty(2, dtype=np.float32)
    y[0] = self._y_buf[0]
    y[1] = self._y_buf[1]
    return y

cpdef void reset(self):
    DFC_Controller_Init(&self._state, 0.00005f) # re-init with same dt

```

4.3 Build Pipeline

The build pipeline converts the .pyx file into a native .pyd (Windows) or .so (Linux) extension. The scripts `build_dfc_controller.bat` and `build_dfc_controller.sh` automate the full sequence:

```

# What the build script does, in order:
1. Clean stale build artifacts:
   del dfc_controller_wrapper*.pyd      <- CRITICAL: stale .pyd takes silent precedence
   del dfc_controller_wrapper*.c        <- the Cython-generated C intermediate
   rmdir /s /q build                    <- setuptools build directory

2. Run Cython transpiler (via setuptools):
   python setup_dfc_controller.py build_ext --inplace
   # .pyx -> .c (Cython-generated C; 10-50x larger than .pyx)
   # .c -> .pyd/.so (C compiler: MSVC on Windows, GCC/Clang on Linux)

3. Copy output to parent directory:
   copy dfc_controller_wrapper.cp312-win_amd64.pyd dfc_controller_wrapper.pyd
   # The ABI-tagged name changes with Python version; the plain name is stable

# The setup_*.py file:
from setuptools import setup, Extension
from Cython.Build import cythonize
import numpy as np

ext = Extension(
    "dfc_controller_wrapper",
    sources=["dfc_controller_wrapper.pyx",
             "embed_sim_dfc_controller.c"], # C source compiled in directly
    include_dirs=[np.get_include(), "."],
    extra_compile_args=["-O2", "-march=native"],
)

```

Stale build is the most common silent failure: Setuptools uses file timestamps to decide whether to recompile. If a .pyx file is modified but the .pyd is newer (e.g. because you restored from backup, changed the timestamp, or copied the file), setuptools silently skips the rebuild. The old .pyd loads, presents the old behaviour, and the test passes for the wrong reason. Always delete the .pyd and .c before rebuilding.

4.4 Loading the Wrapper at Runtime

The wrapper is loaded lazily in `_load_wrapper()`, called from `__init__` only when `use_c_backend=True`. The load uses `importlib.util.spec_from_file_location()` rather than a plain `import` statement, because the `.pyd/.so` lives in a directory that is not always on `sys.path`:

```
def _load_wrapper(self):
    pyd_path = _C_SRC / "dfc_controller_wrapper" # no extension
    try:
        # importlib.util handles .pyd on Windows and .so on Linux
        spec = importlib.util.spec_from_file_location(
            "dfc_controller_wrapper",
            str(pyd_path.with_suffix(".pyd" if os.name=="nt" else ".so")))
        module = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(module)
        self._wrapper = module.DFCControllerWrapper(
            dt_s = self.dt_s,
            # ... all constructor args from instance attributes
        )
    except (FileNotFoundError, ImportError) as e:
        raise RuntimeError(
            f"dfc_controller_wrapper not found. Build with:
            "
            f" cd {_C_SRC}
            "
            f" python setup_dfc_controller.py build_ext --inplace"
        ) from e
```

Part 5 StepGenerator: How C is Emitted

5.1 What StepGenerator Does

`StepGenerator` takes a verified, simulated block graph and emits two files: `EmbedSim_step.h` and `EmbedSim_step.c`. The generated code is a faithful translation of the Python simulation -- same algorithm, same numerical parameters, same execution order -- but expressed in MISRA C:2012.

`StepGenerator` is called after `sim.run()` completes. The block instances are live and have their final attribute values. This is important for `C_INIT_ARGS`: the emitted `Init()` literals match the simulation parameters exactly because they are read from the same Python objects that ran the simulation.

5.2 DFS Backward Collection

`StepGenerator`'s constructor calls `_collect_blocks()`, which performs a DFS backward from `cg_end` to `cg_start` collecting all blocks in the CodeGen region:

```
def _collect_blocks(self):
    # DFS backward from cg_end, stop at cg_start
    # Result: self.blocks = list in topological order (ready for C emission)
    visited = set()
    order = []

    def dfs(block):
        if block in visited:
            return
        visited.add(block)
        for inp in block.inputs:
            if inp is self.cg_start:
                order.append(inp) # cg_start is the base case
                return
            if not getattr(inp, "C_CODEGEN_EXCLUDE", False):
                dfs(inp)
        order.append(block)

    dfs(self.cg_end)
    # order is in dependency-resolved sequence: cg_start first, cg_end last
    self.blocks = order
```

5.3 Header Generation: `EmbedSim_step.h`

The header file defines everything the MCU integration layer needs to call `EmbedSim_Step()`. It contains: the `EMBEDSIM_DT` macro, all `#include` directives for block headers, the input struct, the output struct, and the `Init` and `Step` prototypes.

`EMBEDSIM_DT` macro

```
# dt_hz is passed as 1.0/DT = 20000.0 for the 20 kHz DFC simulation
# StepGenerator emits:
#define EMBEDSIM_DT (0.00005000f) /* 1 / 20000.0 Hz = 50 us */
#          ^^^^^^^^^^^^^^^
#          Python: f"{1.0/dt_hz:.8f}" <- always 8 decimal places
```

`#include` directives -- derived from `C_HEADERS`

```
# StepGenerator iterates all blocks in the CodeGen region
# For each block, for each entry in block.C_HEADERS:
#   emit: #include "<entry.lower()>"
```

```
# DFControllerBlock.C_HEADERS = ["embed_sim_dfc_controller.h", "embed_sim_dfc_gains.h"]
# SVPWMBlock.C_HEADERS       = ["svpwm.h"]
# SpeedRampBlock.C_HEADERS   = ["motor_utility_blocks.h"]

# Generated header section:
#include "embed_sim_dfc_controller.h"
#include "embed_sim_dfc_gains.h"
#include "svpwm.h"
#include "motor_utility_blocks.h"
# Note: duplicates are automatically deduplicated by the set() in _collect_headers()
```

EmbedSim_Input_T -- assembled from upstream INPUT_NAMES

The input struct is built by `_in_sigs()`: iterate blocks in reverse order from `cg_start`, collect any block that has `INPUT_NAMES` set, and map each name to a struct field using `INPUT_KEEP` to select which upstream output indices are exposed.

```
# In db42s02_closed_loop_dfc_20k.py:
# ctrl_packer outputs: [omega_ref_mech, theta_m, ia, ib, ic]
# DFControllerBlock declares:
INPUT_NAMES = ["omega_ref_mech", "theta_m", "ia", "ib", "ic"]
INPUT_KEEP  = [0, 1, 2, 3, 4]      # all 5 signals pass through
C_FIELD_COMMENTS = {
    "omega_ref_mech": "Mechanical speed reference [rad/s]",
    "theta_m":       "Encoder mechanical angle [rad], accumulating",
    "ia":            "Phase-A current [A]",
    "ib":            "Phase-B current [A]",
    "ic":            "Phase-C current [A]",
}

# Generated struct:
typedef struct {
    real32_T omega_ref_mech; /* Mechanical speed reference [rad/s] */
    real32_T theta_m;       /* Encoder mechanical angle [rad], accumulating */
    real32_T ia;            /* Phase-A current [A] */
    real32_T ib;            /* Phase-B current [A] */
    real32_T ic;            /* Phase-C current [A] */
} EmbedSim_Input_T;
#
#      ^^^^^^^^^^^^^
#      field name comes directly from INPUT_NAMES[i]
#      always real32_T unless C_INPUT_TYPES overrides
```

EmbedSim_Output_T -- assembled from downstream OUTPUT_NAMES

```
# SVPWMBlock declares:
OUTPUT_NAMES = ["ta", "tb", "tc", "sector"]
OUTPUT_KEEP  = [0, 1, 2, 3]
C_OUTPUT_TYPES = {"sector": "uint8_T"} # overrides default real32_T for this field

# Generated struct:
typedef struct {
    real32_T ta; /* Phase A duty cycle [0, 1] */
    real32_T tb; /* Phase B duty cycle [0, 1] */
    real32_T tc; /* Phase C duty cycle [0, 1] */
    uint8_T sector; /* Active SVPWM sector 0..5 */
#
#      ^^^^^
#      StepGenerator reads C_OUTPUT_TYPES.get("sector", "real32_T")
#      and uses it instead of the default
} EmbedSim_Output_T;
```

5.4 Source Generation: EmbedSim_step.c

The source file contains: static state declarations, `EmbedSim_Init()`, and `EmbedSim_Step()`. The Init and Step bodies are built by iterating `self.blocks` in topological order and calling `_emit_block()` for each.

Static state declarations -- `block.state_struct + block.name`

```
# StepGenerator iterates blocks and emits:
# for block in self.blocks:
#     if block.state_struct is not None:
#         emit: "static {block.state_struct} {block.name}_state;"

# Generated declarations:
static DFC_State_T      dfc_state;
static SpeedRamp_T      speed_ref_state;
static SVPWM_State_T    svpwm_state;
#     ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^
#     state_struct      block.name + "_state"
# "static" enforces MISRA Rule 8.7: internal linkage for file-scope objects
```

EmbedSim_Init() -- typed literal arguments from `C_INIT_ARGS`

```
# StepGenerator.generate_init():
# for block in self.blocks:
#     if block.init_func is not None:
#         args = [f"{getattr(block, a):.8f}" for a in block.C_INIT_ARGS]
#         emit: "{block.init_func}(&{block.name}_state, {'', '.join(args)});"

void EmbedSim_Init(void) {
    DFC_Controller_Init(&dfc_state, 0.00005000f);
#     ^^^^^^^^^^^^^^^^^
#     from dfc.dt_s = 50e-6

    SpeedRamp_Init(&speed_ref_state, 41.88790205f, 0.30000000f);
#     ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^
#     omega_target ramp_time
#     (live instance attributes read at codegen time)
}
```

`_emit_block()` -- the decision tree

For each block, `_emit_block()` applies a decision tree in priority order. The first matching condition determines what is emitted:

Priority	Condition	What is emitted	Example
1	<code>C_CUSTOM_EMIT</code> is set	Verbatim C string from <code>C_CUSTOM_EMIT</code> attribute	DFCControllerBlock: DFC_Controller_Step() + normalisation
2	<code>IS_UNIT_DELAY</code> is True	memcpy from previous-step <code>y[]</code> buffer (z^{-1} register)	VectorDelay blocks inside CodeGen region
3	<code>NUM_INPUTS == 0</code>	Source block pattern: Step(&state, dt, y) with no <code>u[]</code> array	SpeedRampBlock, VectorConstant
4	<code>step_func</code> is set	Normal pattern: build <code>u[]</code> then call Step(&state, u, dt, y)	SVPWMBlock, coordinate transforms

Priority	Condition	What is emitted	Example
5	C_SOURCES set, no step_func	TODO comment in generated code	Incomplete block: needs step_func
6	None of the above	Python-only comment: block has no C implementation	Diagnostic / logging blocks

Normal block emission -- building u[] from upstream y[] buffers

```
# StepGenerator._emit_block() for a normal block (priority 4):
# 1. Declare y[] output buffer
real32_T y_svpwm_pack[3]; /* y_<block.name>[<OUTPUT_SIZE>] */

# 2. Build u[] input buffer from upstream blocks
real32_T u_svpwm_pack[2]; /* u_<block.name>[sum of upstream OUTPUT_SIZES] */
u_svpwm_pack[0] = y_dfc[0]; /* v_alpha from DFC block */
u_svpwm_pack[1] = y_dfc[1]; /* v_beta from DFC block */
#
#          ^^^^^^^^^^
#          upstream.name + "[" + index + "]"
#          index from upstream.OUTPUT_KEEP if set, else sequential

# 3. Call the step function
SVPWMPack_Step(&svpwm_pack_state, u_svpwm_pack, EMBEDSIM_DT, y_svpwm_pack);
#          ^^^^^^^^^^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^^^^
#          block.name+"_state" "u_"+block.name          "y_"+block.name
```

C_CUSTOM_EMIT -- when and why it exists

The `C_CUSTOM_EMIT` attribute is an escape hatch for blocks whose C function has a non-standard ABI -- one that `StepGenerator` cannot automatically handle with the standard `Step(&state, u, dt, y)` pattern.

`DFCControllerBlock` is the canonical example: its C function outputs physical voltages [V], but `SVPWMPackBlock` expects normalised references in [-1,+1]. The normalisation `y_dfc[0] = y_dfc_out.v_alpha / DFC_V_MAX` cannot be expressed as a generic `Step()` call.

```
# DFCControllerBlock.C_CUSTOM_EMIT:
C_CUSTOM_EMIT = """
/* --- dfc_controller (DFCControllerBlock) --- */
/* DFC_Controller_Step() outputs physical voltages [V]. */
/* The SVPWM block expects normalised [-1,+1] references. */
/* Divide by DFC_V_MAX = DFC_V_DC / sqrt(3) before SVPWM. */
DFC_Input_T   u_dfc;
DFC_Output_T  y_dfc_out;
real32_T      y_dfc[2];

u_dfc.omega_ref_mech = in->omega_ref_mech; /* <- reads from Input_T */
u_dfc.theta_m        = in->theta_m;
u_dfc.ia              = in->ia;
u_dfc.ib              = in->ib;
u_dfc.ic              = in->ic;

DFC_Controller_Step(&dfc_state, &u_dfc, dt, &y_dfc_out);

/* Normalise: physical [V] -> SVPWM [-1, +1] */
y_dfc[0] = y_dfc_out.v_alpha / DFC_V_MAX;
y_dfc[1] = y_dfc_out.v_beta / DFC_V_MAX;
"""
# Note: C_CUSTOM_EMIT is responsible for declaring y_dfc[2]
# so the next block (SVPWMPackBlock) can reference y_dfc[0] and y_dfc[1]
```

Rule: C_CUSTOM_EMIT is not a workaround, it is a contract: When you use C_CUSTOM_EMIT, you take full responsibility for: declaring the y_<name>[] output buffer so downstream blocks can read it, reading from in-> for Input_T fields, writing correct units (the downstream block expects a specific unit convention), and maintaining MISRA C:2012 compliance. If StepGenerator's standard path could handle your block, it should. Every unnecessary C_CUSTOM_EMIT is a maintenance liability.

Pack section -- writing EmbedSim_Output_T

After all blocks have been emitted, StepGenerator emits the pack section: for each OUTPUT_NAMES entry on the final output block, read the corresponding index from the output block's y_ buffer and write it to the output struct field, applying the C type cast specified by C_OUTPUT_TYPES:

```
/* Pack section -- written by _emit_pack() */
out->ta      =      y_svpwm[0]; /* real32_T: default */
out->tb      =      y_svpwm[1]; /* real32_T: default */
out->tc      =      y_svpwm[2]; /* real32_T: default */
out->sector  = (uint8_T)y_svpwm[3]; /* uint8_T: from C_OUTPUT_TYPES["sector"] */
#           ^^^^^^^
#           StepGenerator emits cast only when C_OUTPUT_TYPES overrides
#           the default real32_T for that field
```

Unpack section -- reading EmbedSim_Input_T

The first thing EmbedSim_Step() does is read from in-> and place the values into the y[] buffers of the first blocks in the CodeGen region. These blocks then appear to have outputs already computed when downstream blocks are emitted:

```
void EmbedSim_Step(float dt, const EmbedSim_Input_T *in, EmbedSim_Output_T *out) {

    /* Unpack section -- built by _emit_unpack() */
    /* ctrl_packer block provides the Input_T signals */
    real32_T y_ctrl_packer[5];
    y_ctrl_packer[0] = in->omega_ref_mech; /* INPUT_KEEP[0] = 0 */
    y_ctrl_packer[1] = in->theta_m;        /* INPUT_KEEP[1] = 1 */
    y_ctrl_packer[2] = in->ia;              /* INPUT_KEEP[2] = 2 */
    y_ctrl_packer[3] = in->ib;              /* INPUT_KEEP[3] = 3 */
    y_ctrl_packer[4] = in->ic;              /* INPUT_KEEP[4] = 4 */
    /* Now downstream blocks can reference y_ctrl_packer[i] normally */

    /* Block emissions follow in topological order ... */
```

Part 6 FMU and Modelica: Physics as Code

6.1 Why FMU is a First-Class Citizen

The Modelica/FMU layer is not a bolted-on extension. It occupies the same architectural role in EmbedSim that the C layer occupies: a technology with a precise, non-overlapping responsibility. The C layer executes the controller. The FMU layer executes the plant. These two things are symmetric: both are compiled artifacts, both are driven by automatically-generated Python wrappers, and both are invisible to the simulation wiring code.

The symmetry is important: the engineer who writes the Modelica model never sees FMPy. The engineer who writes the controller never sees the FMU binary. Each writes in their natural language (Modelica equations, Python block logic) and the framework handles the glue.

6.2 The Complete FMU Pipeline

```

Step 1: Write PMSM_Plant_FMU.mo (Modelica source)
Physical PMSM model: dq-frame voltage equations + torque + mechanics
Standard FMI 2.0 connectors: input/output/parameter declarations

Step 2: Compile to PMSM_Plant_FMU.fmu (OpenModelica: omc)
.fmu is a ZIP archive containing:
  modelDescription.xml  <- interface definition (inputs, outputs, params)
  binaries/linux64/*.so <- compiled Modelica C code
  sources/PMSM_Plant_FMU.c <- generated C (readable, not hand-written)
  resources/*.bin       <- Jacobian matrices for DASSL solver

Step 3: Run gen_fmu.py (one-shot, run whenever .mo changes)
Reads modelDescription.xml from .fmu (or .mo via MoParser)
Calls ClientGenerator to emit PMSM_Plant_FMUBlock.py
-> VectorBlock subclass with INPUT_VARS, OUTPUT_VARS, DEFAULT_PARAMS

Step 4: Simulation uses PMSM_Plant_FMUBlock as the plant
FMPy wraps the .so binary: set_real(inputs), do_step(t, dt), get_real(outputs)
EmbedSim calls this once per simulation step
C_CODEGEN_EXCLUDE = True: never enters StepGenerator

```

6.3 modelDescription.xml -- The FMU Interface Contract

The modelDescription.xml inside the .fmu ZIP is the FMI 2.0 standard interface contract. It is the only file that gen_fmu.py needs to read -- the binary .so can be replaced or recompiled without changing the Python wrapper as long as the interface contract does not change.

XML element	What it contains	Mapped to
modelName	PMSM_Plant_FMU	VectorBlock class name: PMSM_Plant_FMUBlock
ScalarVariable causality=input	duty_a, duty_b, duty_c, v_dc, T_load	FMUBlock.INPUT_VARS list
ScalarVariable causality=output	rpm, ia, ib, ic, theta_m, T_em, id_out, iq_out	FMUBlock.OUTPUT_VARS list
ScalarVariable causality=parameter	R, L_d, L_q, lambda_pm, J, B_fric, p, v_dc_nom	FMUBlock.DEFAULT_PARAMS dict
Real start= attribute	0.19, 1.25e-4, 0.0014 ...	DEFAULT_PARAMS values

6.4 gen_fmu.py -- How the Wrapper is Generated

`gen_fmu.py` uses two internal classes. `MoParser` reads the `.mo` Modelica source as plain text (no Modelica compiler needed) and extracts the interface. `ClientGenerator` renders the `VectorBlock` subclass using Python string templates.

```
# gen_fmu.py in full:
parser = MoParser(mo_text)          # parse PMSM_Plant_FMU.mo
model  = parser.parse()             # model.name, model.inputs, model.outputs,
model.parameters

generator = ClientGenerator(model)
generated_code = generator.generate()

# The generated class (PMSM_Plant_FMUBlock.py):
class PMSM_Plant_FMUBlock(FMUBlock):
    INPUT_VARS = ['duty_a', 'duty_b', 'duty_c', 'v_dc', 'T_load']
    OUTPUT_VARS = ['rpm', 'ia', 'ib', 'ic', 'theta_m', 'T_em', 'id_out', 'iq_out']
    DEFAULT_PARAMS = {
        'R': 0.19,
        'L_d': 0.000125,
        'L_q': 0.000125,
        'lambda_pm': 0.0014,
        'J': 2.4e-6,
        'B_fric': 1e-6,
        'p': 4,
        'v_dc_nom': 17.0
    }

    def __init__(self, name, fmu_path):
        super().__init__(name=name, fmu_path=fmu_path,
                        input_names=self.INPUT_VARS,
                        output_names=self.OUTPUT_VARS,
                        parameters=self.DEFAULT_PARAMS,
                        instance_name=name)
```

6.5 FMUBlock Base Class -- The FMPy Integration

`FMUBlock` is the EmbedSim base class for all FMU-based plant blocks. It wraps FMPy's `FMU2Slave` API into the standard `compute_py()` interface that the simulation engine calls. The FMU runs in co-simulation mode (FMI 2.0 CS): EmbedSim provides inputs, triggers a time step, reads outputs.

```
class FMUBlock(VectorBlock):
    C_CODEGEN_EXCLUDE = True    # FMU always stays outside CodeGen boundary

    def compute_py(self, t, dt, input_values):
        # 1. Unpack VectorSignal -> flat float list
        values = [float(v) for v in input_values[0].value]

        # 2. Set FMU inputs via FMPy
        self._fmu.setReal(self._input_vrefs, values)
        #             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        #             value references from modelDescription.xml
        #             looked up by name during __init__

        # 3. Advance FMU by one timestep
        self._fmu.doStep(t, dt)

        # 4. Read FMU outputs
        outputs = self._fmu.getReal(self._output_vrefs)

        # 5. Wrap in VectorSignal
        self.output = VectorSignal(np.array(outputs, dtype=DEFAULT_DTYPE), self.name)
        return self.output
```

6.6 PMSM_Python_Plant -- Portable FMU Alternative

When the FMU binary is not available (different OS, ARM target, unit testing), `PMSM_Python_Plant` provides an identical interface using a pure-Python RK4 integration of the dq-frame PMSM equations. The class has `C_CODEGEN_EXCLUDE = True` and the same 8-element output bus [rpm, ia, ib, ic, theta_m, T_em, id, iq]. Controller code runs against either plant without modification.

Key design principle: all coordinate transforms inside `PMSM_Python_Plant` are delegated to `coordinate_transform_blocks.py` -- the same Python mirrors of the canonical C functions. There is no inline transform math anywhere in the plant. This means the Python plant uses the same transform conventions as the controller, preventing sign or scaling mismatches.

```
# PMSM_Python_Plant uses transform blocks, not inline math:
self._ct_clarke      = ClarkeTransformBlock("_plant_clarke",      use_c_backend=False)
self._ct_park        = ParkTransformBlock("_plant_park",         use_c_backend=False)
self._ct_inv_park     = InvParkTransformBlock("_plant_inv_park",  use_c_backend=False)
self._ct_inv_clarke  = InvClarkeTransformBlock("_plant_inv_clarke", use_c_backend=False)

# In _vdq_from_duties():
ab = self._ct_clarke.compute_py(0.0, 0.0, [inp])
dq = self._ct_park.compute_py(0.0, 0.0, [ab_sig, th_sig])
# Same Clarke/Park code path as the controller
# Transform bugs manifest identically in both plant and controller -> easy to spot
```

Part 7 Three Examples: From Engine to Full Production

Example 1 `example_algebraic_loop.py` -- Engine Fundamentals

The purpose of this example is to make the engine's internal mechanics visible. Read the Part 3 docstring in the example file alongside this section.

What the Example Shows You

Three things that are invisible in FOC examples because they are handled automatically:

- **DFS loop detection:** the engine rejects an invalid diagram at construction time, before any computation. You see the exact `ValueError` and understand why it fires.
- **VectorDelay mechanics:** you see the exact execution order (delay first, before its upstream gain), understand why this is correct, and observe the zero-fallback warning at $t=0$.
- **Heun integration:** the recurrence $y(k) = K * [A * \sin(\dots) + y(k-1)]$ is a first-order IIR. You can verify the stability condition $|K| < 1$ by trying $K=0.5$ (stable) and $K=1.1$ (diverges). The `ODESolver` choice does not affect stability for this system, but affects accuracy for nonlinear systems.

The Diagnostic Pattern

Part 3 of the example prints a step-by-step trace of what the engine does at $t=0$ and $t=0.01$. This is the most direct way to understand `_compute_all_blocks()`. The trace shows exactly which block runs, what its inputs are, and what its output becomes. When you encounter unexpected simulation behaviour in FOC examples, adding this kind of trace (via `SimulationLogger`) is the first debugging step.

Example 2 `db42s02_openloop_fm.py` -- CodeGen Boundary + FMU

The purpose of this example is to make the CodeGen boundary visible and to show the FMU plant in a complete simulation loop.

The Boundary in Practice

In this example, the boundary is placed after `SVPWMPackBlock`. This is intentional: the AURIX ISR can compute `sqrtf()` and `atan2f()` faster than EmbedSim can represent them as `VectorBlocks`. The simulation verifies the full signal chain including the Cartesian-to-polar conversion; only the SVPWM sector calculation is pushed to the MCU.

Moving the boundary is a one-line wiring change. To move it earlier (absorb `InvPark` into the MCU):

```
# Original Rev 3.0 boundary:
inv_park >> svpwm_pack >> cg_start >> svpwm >> cg_end

# Future Rev 4.0 boundary (InvPark inside CodeGen):
inv_park >> cg_start >> svpwm_pack >> svpwm >> cg_end
# EmbedSim_Input_T now needs: { v_alpha, v_beta } instead of { magnitude, angle_rad }
# Only the wiring changes. InvPark block implementation does not change.
```

What `DB42S02PlantBlock` Teaches About `C_CODEGEN_EXCLUDE`

The `DB42S02PlantBlock` subclass is the cleanest example of the exclude mechanism. Notice that it:

- sets `C_CODEGEN_EXCLUDE = True` -- it is a simulation-only object
- receives the same `[ta, tb, tc, sector]` bus that the AURIX ISR will produce

- injects `V_DC = 17.0` and `T_LOAD = 0.01` as physical constants -- these are never CodeGen signals
- re-packs a stable 5-element scope bus `[rpm, ia, ib, ic, T_em]` -- independent of FMU's 8-element output order

This pattern teaches you how to write **any** simulation-only block: make it `C_CODEGEN_EXCLUDE`, give it the same input interface as the real hardware, inject physical constants internally, and expose a stable, well-documented scope bus.

Reading the Topology HTML

The example calls `sim.topo.export_html()` to produce an interactive block diagram. The HTML file shows the CodeGen boundary visually: blocks inside the boundary are highlighted, blocks outside are greyed. Wire labels (set via `_wire_labels` dict in the example) annotate every signal with its physical meaning and units. This HTML is the single best tool for verifying that the block diagram matches your intent before running the simulation.

Example 3 `db42s02_closed_loop_dfc_20k.py` -- Full Production Stack

This is the complete production EmbedSim application. Every mechanism described in Parts 1-6 appears here in final form. Read it as a verification that the architecture works.

The Feedback Loop as VectorDelay at Scale

In Example 1, VectorDelay broke a 2-block feedback loop. Here it breaks an 8-signal bus feedback loop spanning the entire controller chain. The principle is exactly the same; only the scale changes.

```
# The feedback topology:
motor >> motor_delay >> ctrl_packer >> cg_start >> dfc >> svpwm_pack >> svpwm >> cg_end >>
motor
#           ^
#           |
#           +------(z^-1)-----[8 signals: rpm, ia, ib, ic, theta_m, Tem, id, iq]-----+
#
# DFS execution order (motor_delay is a LoopBreaker):
#   motor_delay, ctrl_packer, cg_start, dfc, svpwm_pack, svpwm, cg_end, motor
#
# At step k:
#   motor_delay outputs motor(k-1) <- stored from previous step
#   ctrl_packer uses motor(k-1) to build the DFC input bus
#   dfc computes v_alpha, v_beta from motor(k-1) state
#   svpwm computes duties from v_alpha, v_beta
#   motor advances using duties(k) -> produces motor(k)
#   motor_delay latches motor(k) for step k+1
```

The Noise Model as a Plant Decorator

The AURIX noise model is implemented entirely inside `DB42S02PlantBlock.compute_py()` as a post-processing step on the plant's outputs. This is the correct architectural place: noise is a hardware property of the sensor chain, not a property of the controller. The controller receives the noisy signals and must handle them; the simulation verifies that it does.

The noise model calls helper functions (`_adc_noise`, `_pwm_spike`, `_enc_quantise`, `_enc_glitch`, `_deadtime_disturbance`, `_vdc_ripple`) each of which models one specific hardware non-ideality. Each is independently testable by setting `with_noise=False` in `_run_sim()` -- the clean plant runs the same controller without any noise, providing a baseline for performance comparison.

The NN Tuner as a Simulation Consumer

The NN surrogate tuner is the best example of EmbedSim being used as a platform, not just a tool. The tuner calls `_run_sim()` 40+ times in Phase 1 (LHS exploration), each call running the complete 5-second closed-loop simulation. It consumes simulation results as training data for the MLP surrogate, then optimises the surrogate to find better gains, then calls `_run_sim()` one more time to verify the recommended gains.

The key architectural point: the tuner is not special. It is ordinary Python code that calls the same `_run_sim()` function as the manual test. The simulation is a pure Python function that takes parameters and returns a result dict. This composability -- simulation as function -- is what makes automated gain optimisation, Monte Carlo analysis, and hardware-in-the-loop testing all natural extensions of the same simulation code.

CodeGen in the DFC Example -- What is Different from Example 2

In Example 2, the CodeGen region contained only SVPWMBlock. In this example it contains the full DFC controller. The generated `EmbedSim_Step()` now includes:

- Unpack: `y_ctrl_packer[]` from `in->omega_ref_mech ... in->ic`
- C_CUSTOM_EMIT: `DFC_Controller_Step()` called with `DFC_Input_T` / `DFC_Output_T` structs + voltage normalisation
- Normal: `SVPWMPack_Step()` called with `y_dfc[]` as input
- Normal: `SVM_CalculateDutyCycle()` called with `y_svpwm_pack[]` as input
- Pack: `out->ta` / `out->tb` / `out->tc` / `out->sector` from `y_svpwm[]`

The input struct changed from `{ magnitude, angle_rad }` (Rev 3.0 open-loop) to `{ omega_ref_mech, theta_m, ia, ib, ic }` (DFC closed-loop). This boundary change required: one different `CodeGenStart` position in the wiring; `INPUT_NAMES` / `INPUT_KEEP` set on `DFCControllerBlock`; nothing else.

Part 8 Reference

8.1 Complete VectorBlock Metadata Reference

Attribute	Type	Set by	Read by	Description
PYX_FILE	str	Block author	PYXInspector at import	Path to .pyx; triggers auto-population
step_func	str	PYXInspector or explicit	StepGenerator	C step function name
state_struct	str	PYXInspector or explicit	StepGenerator	C state struct type
init_func	str	PYXInspector or explicit	StepGenerator	C init function name
C_SOURCES	list	Block author	StepGenerator (compilation)	.c files compiled into firmware
C_HEADERS	list	Block author	StepGenerator (emit #include)	Headers, auto-lowercased
NUM_INPUTS	int	Block author	StepGenerator	0 = source block, N = normal
OUTPUT_SIZE	int	Block author	StepGenerator	Length of y[] output buffer
C_INIT_ARGS	list	Block author	StepGenerator	Instance attr names -> Init() literals
C_CUSTOM_EMIT	str	Block author	StepGenerator	Verbatim C for non-standard ABI
C_CODEGEN_EXCLUDE	bool	Block author	StepGenerator	True = never emitted
INPUT_NAMES	list	Block author or instance	StepGenerator	Input_T field names
INPUT_KEEP	list	Block author or instance	StepGenerator	Upstream y[] indices for Input_T
C_INPUT_TYPES	dict	Block author	StepGenerator	C type per Input_T field
OUTPUT_NAMES	list	Block author or instance	StepGenerator	Output_T field names
OUTPUT_KEEP	list	Block author or instance	StepGenerator	y[] indices for Output_T
C_OUTPUT_TYPES	dict	Block author	StepGenerator	C type per Output_T field
C_FIELD_COMMENTS	dict	Block author	StepGenerator	/* comments */ for Input_T fields
TOPO_CATEGORY	str	Block author	TopologyPrinter	Visual category in HTML diagram

8.2 Generated C Name Derivation Summary

Generated C name	Derivation rule	Concrete example
State variable	<code>block.name + "_state"</code>	<code>dfc -> dfc_state</code>
Output buffer	<code>"y_" + block.name</code>	<code>y_dfc</code>
Input buffer	<code>"u_" + block.name</code>	<code>u_svpwm_pack</code>
Init call	<code>init_func("&" + name + "_state", *C_INIT_ARGS_values)</code>	<code>DFC_Controller_Init(&dfc_state, 0.00005f)</code>
Step call	<code>step_func("&" + name + "_state", u, dt, y)</code>	<code>DFC_Controller_Step(&dfc_state, ...)</code>
Input_T field	<code>INPUT_NAMES[i]</code>	<code>omega_ref_mech, theta_m, ia</code>
Output_T field	<code>OUTPUT_NAMES[i]</code>	<code>ta, tb, tc, sector</code>
Output_T type	<code>C_OUTPUT_TYPES.get(name, "real32_T")</code>	<code>sector -> uint8_T</code>
#include path	<code>C_HEADERS[i].lower()</code>	<code>embed_sim_dfc_controller.h</code>
Timestep macro	<code>1.0/dt_hz</code> formatted as <code>.8f + "f"</code>	<code>0.00005000f</code>
MISRA f-suffix	Applied to all <code>Init()</code> float literals	<code>41.88790205f</code> not <code>41.88790205</code>

8.3 Glossary

Term	Definition
ATOM	Advanced Timer Output Module -- GTM sub-module, 24-bit, CMU clock, ARU-connected.
C_CODEGEN_EXCLUDE	Block attribute. True -> block never enters StepGenerator.
C_CUSTOM_EMIT	Verbatim C fragment for non-standard ABI. Responsible for declaring <code>y_<name>[]</code> buffer.
C_INIT_ARGS	List of instance attr names read at codegen time and emitted as typed f-suffixed float literals.
ClientGenerator	<code>gen_fmu.py</code> class: renders VectorBlock subclass Python code from a parsed Modelica model.
CodeGen boundary	The region between <code>CodeGenStart</code> and <code>CodeGenEnd</code> translated to C by StepGenerator.
DFC	Differential Flatness Controller -- model-based feedforward law cancelling plant dynamics.
DEFAULT_DTYPE	<code>numpy.float32</code> -- all EmbedSim signal arrays use this dtype.
FMI 2.0	Functional Mock-up Interface 2.0 -- the standard API for FMU co-simulation.
FMPy	Python library wrapping FMI 2.0 <code>.so/.dll</code> binaries via ctypes.
FMU	Functional Mock-up Unit -- Modelica model compiled to a self-contained ZIP binary.

Term	Definition
float[:,1]	Cython C-contiguous float32 memoryview. Mandatory in all set_inputs(). Silently zeroes on float64 input.
gen_fmu.py	One-shot utility: reads .mo or modelDescription.xml, emits VectorBlock FMU wrapper.
LoopBreaker	VectorBlock subclass marker. DFS cuts back-edges at LoopBreaker instances.
MoParser	gen_fmu.py class: parses Modelica .mo source as text to extract inputs/outputs/parameters.
MISRA C:2012	Coding standard. Key rules: 7.2 (f suffix on floats), 8.7 (static linkage), 15.5 (single return).
PYXInspector	Regex-based .pyx parser. Runs at class-definition time via __init_subclass__.
real32_T	typedef float real32_T -- single-precision IEEE 754. Used throughout generated C.
SMO	Sliding Mode Observer -- back-EMF estimation in alphabeta frame for sensorless speed.
SpeedFusion	Adaptive blend of encoder speed and SMO speed, weight = f(omega).
StepGenerator	Emits EmbedSim_step.c/h from the CodeGenStart/End region after DFS collection.
SVPWM	Space Vector PWM -- alphabeta voltage -> three-phase duty cycles.
VectorDelay	z^{-1} LoopBreaker. Outputs $y(k-1)$ at step k . The universal algebraic loop solution.
VectorSignal	Thin wrapper around np.float32 array. The type of every block.output.